

Context Manager - Design

1. Purpose

The Context Manager gives the ADH Copilot Accelerator a shared, governed memory across its 10-phase pipeline. Each specialist agent writes its key decisions into a central state file. Subsequent agents read from it instead of reloading large documents. A read-only broker agent answers factual queries via two-tier retrieval - state store first, targeted file read second.

Two problems this solves:

- **Token cost:** Without shared state, each agent reloads full documents (~8,000 to “15,000 tokens) to answer simple questions. The Context Manager answers most queries in ~50 tokens from the state store.
- **Single source of truth:** All key decisions are recorded in one JSON file. No cache to invalidate, no index to maintain.

2. Components

Component	File	Purpose
State file	.adh-tracker/session-context.json	Shared state for the current pipeline run (source of truth)
Reducer CLI	context.py	Reads/writes state file, enforces schema validation
Broker agent	context-keeper.agent.md	Answers factual queries via two-tier retrieval

All scripts live at: [adh-accelerator/.github/skills/tracker-checkpoint/](#)

Agent lives at: [adh-accelerator/agents/](#)

3. State File - session-context.json

3.1 Schema

```
{  
  "version": "2.0",  
  "run_id": "<custodian>-<dataset>-<yyyymmdd>-<seq>",  
  "meta": {  
    "custodian": "",  
    "dataset": "",  
    "env": "dev",  
    "started_at": "",  
    "last_updated": ""  
  },  
  "shared": {  
    "required_phases": [],  
    "primary_keys": "",  
    "kv_scope": "",  
    "custodian_catalog_prefix": ""  
  },  
  "agents": {  
    "requirements": {  
      "output_file": null,  
      "source_system": null,  
      "source_schema": null,  
      "source_table": null,  
      "source_format": null,  

```

```
"load_type": null,  
  
"watermark_column": null,  
  
"target_layers": [],  
  
"data_classification": null  
},  
  
"hld": {  
  
  "output_file": null,  
  
  "medallion_scope": [],  
  
  "data_flow": null,  
  
  "bronze_catalog": null,  
  
  "bronze_schema": null,  
  
  "std_schema": null  
},  
  
"lld": {  
  
  "output_file": null,  
  
  "silver_catalog": null,  
  
  "silver_schema": null,  
  
  "silver_merge_key": null,  
  
  "gold_schema": null,  
  
  "dimension_tables": [],  
  
  "fact_tables": [],  
  
  "key_decision_set": null  
},  
  
"code_bronze": {  
  
  "output_file": null,
```

```

    "ingestion_config_path": null,

    "job_yaml_path": null

},

"code_silver": { "output_file": null, "notebook_path": null },

"code_gold": { "output_file": null, "notebook_path": null },

"sql_views": { "output_files": [] },

"test": { "test_cases_file": null, "test_scripts": [] },

"review": { "output_file": null, "pr_url": null, "checklist": [] },

"deploy": { "output_file": null, "deploy_timestamp": null, "version_tag": null }

}

}

```

4. Reducer - context.py

4.1 Command reference

Reads

Command	Returns
context.py get <namespace> <key>	Value of agents.<namespace>.<key>. Exits 1 with NOT_FOUND: <namespace>.<key> if absent or null
context.py get-shared <key>	Value of shared.<key>. Exits 1 with NOT_FOUND: shared.<key> if absent
context.py verify	Prints count of set vs null keys. Format: N set, M null. Status: PASS / WARN / FAIL. Lists null keys grouped by namespace

Writes

Command	Behavior
context.py set <namespace> <key> <value>	Writes agents.<namespace>.<key> = value. Updates meta.last_updated
context.py set-shared <key> <value>	Writes shared.<key> = value. Merges - never clears siblings. Updates meta.last_updated

Lifecycle

Command	Behavior
context.py init --run-id <id> --custodian <c> --dataset <d> --env <e>	Creates session-context.json from schema template. Fails if file exists. Writes meta.started_at and meta.last_updated. Creates .adh-tracker/ if absent

Exit codes

Code	Meaning
0	Success
1	Error (key not found, invalid JSON, I/O error)

All errors print a descriptive message to stderr.

4.2 Schema validation rules

Rule	Behavior
Required keys	meta.custodian, meta.dataset, meta.env must be set before any agent writes
Type checking	Values must be strings, arrays, or objects (no mixing). Strict schema per namespace

Null handling	A key can be null (not set) but cannot be deleted. reset-phase sets namespace keys to null
---------------	--

5. Context Manager Agent - context-keeper.agent.md

5.1 Agent spec

Property	Value
Name	ADH Context Keeper
Model	claude-haiku-4.5
Tools	vscode/readFile, vscode/findFiles, vscode/runTerminal
User-invocable	false
Role	Read-only information broker

5.2 Two-tier retrieval

The agent answers every query by executing tiers in order and stopping at the first hit.

Query received

|

+ → Tier 1: context.py get / get-shared ~50 tokens

|

Found → return immediately

|

+ → Tier 2: targeted file read ~500-1500 tokens

Infer file from fixed mapping (see 5.3)

Read section headers + scan for answer

Found → return + call context.py set to cache in state store

Not found → NOT_FOUND response

5.3 File mapping for Tier 2 (fixed - no unbounded scanning)

Question type	File to read	Search for
source_* questions	agents.requirements.ou tput_file	Section headers, "Source" keyword
design / flow questions	agents.hld.output_file	"Design", "Flow", "Architecture"
schema / key questions	agents.lld.output_file	"Schema", "Key", "Decision"
config / job questions	agents.code_bronze.ou tput_file	"Config", "Job", "Ingestion"
notebook questions	agents.code_silver/gold .output_file	"Notebook", "Logic", "Transformation"

5.4 Response formats

Found (Tier 1):

VALUE: <answer>

SOURCE: session-context.json / agents.<namespace>.<key>

TIER: 1

Found (Tier 2):

VALUE: <answer>

SOURCE: <filename>, section: <section-title>

TIER: 2

ACTION_TAKEN: context.py set <namespace> <key> <value> [caches answer for next query]

Not found:

NOT_FOUND: <key> is not available.

NEXT_STEP: Should be set by Phase <XX> (agents.<namespace>). Check if that phase has run.

ACTION: Run Phase <XX> or supply manually: context.py set <namespace> <key> <value>

5.5 When master agent calls Context Manager

Trigger	Query
Session start	Health check - verify all required keys are set (context.py verify)
Before Phase 04	Get source_schema, load_type, watermark_column
Before Phase 05	Get silver_merge_key, silver_schema, primary_keys
Before Phase 07	Get all output file paths for review
After crash recovery	Check which phases completed (context.py verify)

5.6 Hard rules

- Never call context.py set or set-shared - only read
- Never write to any file in the workspace
- Never invoke specialist agents
- Never guess an answer - always cite SOURCE
- If Tier 1 misses, always try Tier 2 (never give up immediately)

6. Agent write protocol

Every specialist agent follows this lifecycle when interacting with the state store:

Phase starts:

context.py init [if first phase] or verify existing store

During phase (for each key decided):

```
context.py set <namespace> <key> <value>
```

```
context.py set-shared <key> <value> [for cross-cutting facts only]
```

Phase ends:

Write phase output file (markdown)

No explicit status tracking. The broker agent infers phase completion by checking if `output_file` is set.

6.1 Write command examples

LLD phase decides silver merge key

```
context.py set lld silver_merge_key "patient_id"
```

Multiple schemas decided

```
context.py set lld silver_schema "sch_curated_patient"
```

```
context.py set lld gold_schema "sch_analytics_patient"
```

Cross-cutting fact (shared with all agents)

```
context.py set-shared primary_keys "patient_id, encounter_id"
```

Output file path recorded

```
context.py set lld output_file "design/adh-llid-output.md"
```

7. File layout

.adh-tracker/

-- session-context.json [source of truth, ~5-“10 KB]

-- .gitignore [exclude .lock files]

adh-accelerator/.github/skills/tracker-checkpoint/

-- context.py [reducer CLI, ~300 lines]

-- tests/ [unit tests]

adh-accelerator/agents/

-- context-keeper.agent.md [broker agent]

8. Engineer interface contracts

For specialist agents

To write a decision:

context.py set <namespace> <key> <value>

echo \$? # 0 = success, 1 = error

To read during development/debugging:

context.py get <namespace> <key>

To check phase health:

context.py verify

For Context Manager agent (read-only)

To get a fact:

context.py get <namespace> <key>

If exits 0: answer in stdout

If exits 1: key not found - try Tier 2 (file read)

To cache a discovered fact:

context.py set <namespace> <key> <value>